CN Orange Problem

SDN Mininet

Vedant Sharma

PES2UG24CS577

Section – 4J

19.04.2026

Github : https://github.com/Websurfingg21/CN-SDN-Orange

Project Report

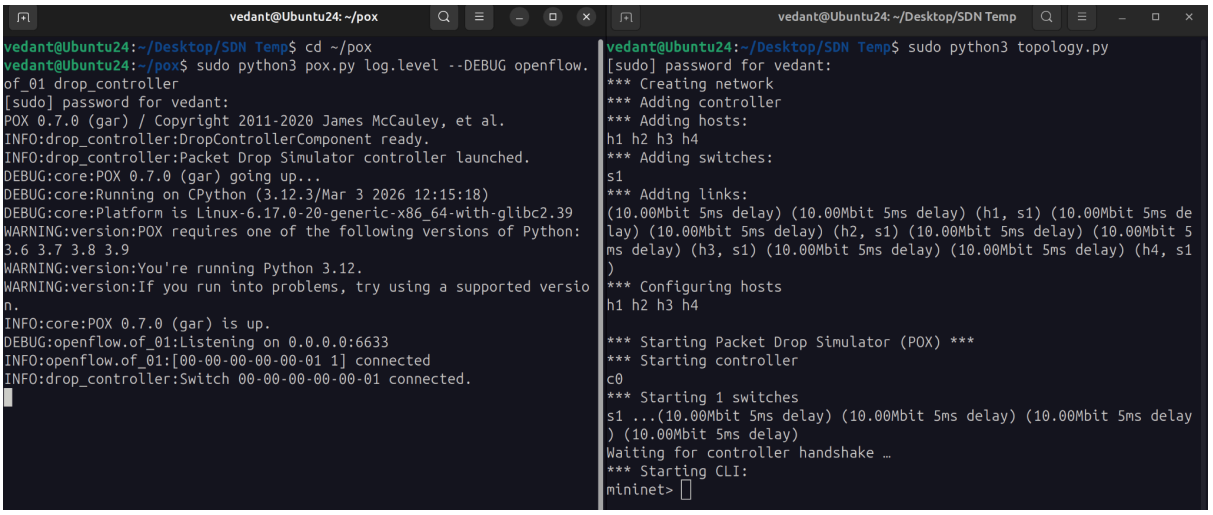
PROCESS & OUTPUT

Starting the controller and topology

Terminal 1 : cd ~/pox

sudo python3 pox.py log.level --DEBUG openflow.of_01 drop_controller

Terminal 2 : sudo python3 topology.py



```
vedant@Ubuntu24: ~/pox
vedant@Ubuntu24:~/Desktop/SDN Temp$ cd ~/pox
vedant@Ubuntu24:~/pox$ sudo python3 pox.py log.level --DEBUG openflow.
of_01 drop_controller
[sudo] password for vedant:
POX 0.7.0 (gar) / Copyright 2011-2020 James McCauley, et al.
INFO:drop_controller:DropControllerComponent ready.
INFO:drop_controller:Packet Drop Simulator controller launched.
DEBUG:core:POX 0.7.0 (gar) going up...
DEBUG:core:Running on CPython (3.12.3/Mar 3 2026 12:15:18)
DEBUG:core:Platform is Linux-6.17.0-20-generic-x86_64-with-glibc2.39
WARNING:version:POX requires one of the following versions of Python:
3.6 3.7 3.8 3.9
WARNING:version:You're running Python 3.12.
WARNING:version:If you run into problems, try using a supported versio
n.
INFO:core:POX 0.7.0 (gar) is up.
DEBUG:openflow.of_01:Listening on 0.0.0.0:6633
INFO:openflow.of_01:[00-00-00-00-00-01 1] connected
INFO:drop_controller:Switch 00-00-00-00-00-01 connected.
```

```
vedant@Ubuntu24: ~/Desktop/SDN Temp
vedant@Ubuntu24:~/Desktop/SDN Temp$ sudo python3 topology.py
[sudo] password for vedant:
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h1, s1) (10.00Mbit 5ms de
lay) (10.00Mbit 5ms delay) (h2, s1) (10.00Mbit 5ms delay) (10.00Mbit 5
ms delay) (h3, s1) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (h4, s1
)
*** Configuring hosts
h1 h2 h3 h4

*** Starting Packet Drop Simulator (POX) ***
*** Starting controller
c0
*** Starting 1 switches
s1 ...(10.00Mbit 5ms delay) (10.00Mbit 5ms delay) (10.00Mbit 5ms delay
) (10.00Mbit 5ms delay)
Waiting for controller handshake …
*** Starting CLI:
mininet>
```

pingall

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Adding Drop Rule and testing

```
mininet> sh ovs-ofctl add-flow s1 "priority=200,ip,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=drop"
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4133ms
```

Wireshark Output during drop rule test

icmp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x2b50, seq=1/256, ttl=64 (no response found!)
2	1.060359659	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x2b50, seq=2/512, ttl=64 (no response found!)
3	2.083055305	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x2b50, seq=3/768, ttl=64 (no response found!)
4	3.108342040	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x2b50, seq=4/1024, ttl=64 (no response found!)
5	4.131713471	10.0.0.1	10.0.0.2	ICMP	98	Echo (ping) request id=0x2b50, seq=5/1280, ttl=64 (no response found!)

Regression Test

```
vedant@Ubuntu24:~/Desktop/SDN Temp$ sudo python3 regression_test.py

============================================================
  Packet Drop Simulator - Regression Test Suite (POX)
============================================================


────────────────────────────────────────────────────────────
  T1 - Baseline: all hosts reachable
────────────────────────────────────────────────────────────
  [PASS]  T1a: h1→h2 baseline 0% loss  (0%)
  [PASS]  T1b: h3→h4 baseline 0% loss  (0%)


────────────────────────────────────────────────────────────
  T2 - Drop rule blocks h1 → h2
────────────────────────────────────────────────────────────
  [PASS]  T2: h1→h2 blocked (100% loss)  (100%)


────────────────────────────────────────────────────────────
  T3 - Non-targeted flows unaffected
────────────────────────────────────────────────────────────
  [PASS]  T3a: h1→h3 still 0% loss  (0%)
  [PASS]  T3b: h2→h4 still 0% loss  (0%)


────────────────────────────────────────────────────────────
  T4 - Removing drop rule restores h1 → h2
────────────────────────────────────────────────────────────
  [PASS]  T4: h1→h2 restored (0% loss)  (0%)
```

```
────────────────────────────────────────────────────────────
  T5 - Multiple simultaneous drop rules
────────────────────────────────────────────────────────────
  [PASS]  T5a: h1→h2 dropped  (100%)
  [PASS]  T5b: h3→h4 dropped  (100%)
  [PASS]  T5c: h1→h3 unaffected  (0%)


────────────────────────────────────────────────────────────
  T6 - Drop rule persists after unrelated traffic
────────────────────────────────────────────────────────────
  [PASS]  T6: h2→h3 rule persists after other traffic  (100%)
```

```
TEST SUMMARY

  ✓  T1a: h1→h2 baseline 0% loss
  ✓  T1b: h3→h4 baseline 0% loss
  ✓  T2: h1→h2 blocked (100% loss)
  ✓  T3a: h1→h3 still 0% loss
  ✓  T3b: h2→h4 still 0% loss
  ✓  T4: h1→h2 restored (0% loss)
  ✓  T5a: h1→h2 dropped
  ✓  T5b: h3→h4 dropped
  ✓  T5c: h1→h3 unaffected
  ✓  T6: h2→h3 rule persists after other traffic

  Result: 10/10 tests passed
================================================================
```

Flow Table

```
[INFO] Flow table (s1):
NXST_FLOW reply (xid=0x4):
 cookie=0x0, duration=38.877s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=38, priority=1,arp,in_port=2,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01,arp_spa=10.0.0.2,arp_tpa=10.0.0.1,arp_op=2 actions=output:1
 cookie=0x0, duration=35.793s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=35, priority=1,arp,in_port=4,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:03,arp_spa=10.0.0.4,arp_tpa=10.0.0.3,arp_op=2 actions=output:3
 cookie=0x0, duration=33.496s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=33, priority=1,arp,in_port=2,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01,arp_spa=10.0.0.2,arp_tpa=10.0.0.1,arp_op=1 actions=output:1
 cookie=0x0, duration=33.495s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=33, priority=1,arp,in_port=1,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02,arp_spa=10.0.0.1,arp_tpa=10.0.0.2,arp_op=2 actions=output:2
 cookie=0x0, duration=30.423s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=30, priority=1,arp,in_port=4,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:03,arp_spa=10.0.0.4,arp_tpa=10.0.0.3,arp_op=1 actions=output:3
 cookie=0x0, duration=30.421s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=30, priority=1,arp,in_port=3,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:04,arp_spa=10.0.0.3,arp_tpa=10.0.0.4,arp_op=2 actions=output:4
 cookie=0x0, duration=28.327s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=28, priority=1,arp,in_port=3,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01,arp_spa=10.0.0.3,arp_tpa=10.0.0.1,arp_op=2 actions=output:1
 cookie=0x0, duration=25.234s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=25, priority=1,arp,in_port=4,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02,arp_spa=10.0.0.4,arp_tpa=10.0.0.2,arp_op=2 actions=output:2
 cookie=0x0, duration=23.256s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=23, priority=1,arp,in_port=3,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01,arp_spa=10.0.0.3,arp_tpa=10.0.0.1,arp_op=1 actions=output:1
 cookie=0x0, duration=23.255s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=23, priority=1,arp,in_port=1,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03,arp_spa=10.0.0.1,arp_tpa=10.0.0.3,arp_op=2 actions=output:3
 cookie=0x0, duration=20.183s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=20, priority=1,arp,in_port=4,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02,arp_spa=10.0.0.4,arp_tpa=10.0.0.2,arp_op=1 actions=output:2
 cookie=0x0, duration=20.182s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=20, priority=1,arp,in_port=2,vlan_tci=0x0000/0x1fff,
dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04,arp_spa=10.0.0.2,arp_tpa=10.0.0.4,arp_op=2 actions=output:4
 cookie=0x0, duration=6.133s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=6, priority=1,arp,in_port=1,vlan_tci=0x0000/0x1fff,dl
_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04,arp_spa=10.0.0.1,arp_tpa=10.0.0.4,arp_op=2 actions=output:4
 cookie=0x0, duration=6.129s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=6, priority=1,arp,in_port=4,vlan_tci=0x0000/0x1fff,dl
_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:01,arp_spa=10.0.0.4,arp_tpa=10.0.0.1,arp_op=2 actions=output:1
 cookie=0x0, duration=4.133s, table=0, n_packets=0, n_bytes=0, idle_timeout=60, idle_age=4, priority=1,arp,in_port=3,vlan_tci=0x0000/0x1fff,dl
_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02,arp_spa=10.0.0.3,arp_tpa=10.0.0.2,arp_op=2 actions=output:2
```

```
 cookie=0x0, duration=38.864s, table=0, n_packets=7, n_bytes=686, idle_timeout=60, idle_age=18, priority=1,icmp,in_port=2,vlan_tci=0x0000/0x1f
ff,dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01,nw_src=10.0.0.2,nw_dst=10.0.0.1,nw_tos=0,icmp_type=0,icmp_code=0 actions=output:1
 cookie=0x0, duration=35.791s, table=0, n_packets=3, n_bytes=294, idle_timeout=60, idle_age=32, priority=1,icmp,in_port=4,vlan_tci=0x0000/0x1f
ff,dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:03,nw_src=10.0.0.4,nw_dst=10.0.0.3,nw_tos=0,icmp_type=0,icmp_code=0 actions=output:3
 cookie=0x0, duration=28.325s, table=0, n_packets=7, n_bytes=686, idle_timeout=60, idle_age=6, priority=1,icmp,in_port=1,vlan_tci=0x0000/0x1ff
f,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03,nw_src=10.0.0.1,nw_dst=10.0.0.3,nw_tos=0,icmp_type=8,icmp_code=0 actions=output:3
 cookie=0x0, duration=28.325s, table=0, n_packets=7, n_bytes=686, idle_timeout=60, idle_age=6, priority=1,icmp,in_port=3,vlan_tci=0x0000/0x1ff
f,dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01,nw_src=10.0.0.3,nw_dst=10.0.0.1,nw_tos=0,icmp_type=0,icmp_code=0 actions=output:1
 cookie=0x0, duration=25.233s, table=0, n_packets=3, n_bytes=294, idle_timeout=60, idle_age=22, priority=1,icmp,in_port=2,vlan_tci=0x0000/0x1f
ff,dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04,nw_src=10.0.0.2,nw_dst=10.0.0.4,nw_tos=0,icmp_type=8,icmp_code=0 actions=output:4
 cookie=0x0, duration=25.232s, table=0, n_packets=3, n_bytes=294, idle_timeout=60, idle_age=22, priority=1,icmp,in_port=4,vlan_tci=0x0000/0x1f
ff,dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02,nw_src=10.0.0.4,nw_dst=10.0.0.2,nw_tos=0,icmp_type=0,icmp_code=0 actions=output:2
 cookie=0x0, duration=6.134s, table=0, n_packets=3, n_bytes=294, idle_timeout=60, idle_age=3, priority=1,icmp,in_port=1,vlan_tci=0x0000/0x1fff
,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04,nw_src=10.0.0.1,nw_dst=10.0.0.4,nw_tos=0,icmp_type=8,icmp_code=0 actions=output:4
 cookie=0x0, duration=6.133s, table=0, n_packets=4, n_bytes=392, idle_timeout=60, idle_age=3, priority=1,icmp,in_port=4,vlan_tci=0x0000/0x1fff
,dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:01,nw_src=10.0.0.4,nw_dst=10.0.0.1,nw_tos=0,icmp_type=8,icmp_code=0 actions=output:1
 cookie=0x0, duration=6.129s, table=0, n_packets=3, n_bytes=294, idle_timeout=60, idle_age=3, priority=1,icmp,in_port=4,vlan_tci=0x0000/0x1fff
,dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:01,nw_src=10.0.0.4,nw_dst=10.0.0.1,nw_tos=0,icmp_type=0,icmp_code=0 actions=output:1
 cookie=0x0, duration=6.129s, table=0, n_packets=3, n_bytes=294, idle_timeout=60, idle_age=3, priority=1,icmp,in_port=1,vlan_tci=0x0000/0x1fff
,dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04,nw_src=10.0.0.1,nw_dst=10.0.0.4,nw_tos=0,icmp_type=0,icmp_code=0 actions=output:4
 cookie=0x0, duration=40.886s, table=0, n_packets=65, n_bytes=4626, idle_age=4, priority=0 actions=CONTROLLER:128
```

Flow Table with Drop Rule

```
mininet> sh ovs-ofctl add-flow s1 "priority=200,ip,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=drop"
mininet> h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4098ms

mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=21.810s, table=0, n_packets=1, n_bytes=42, idle_timeout=30, priority=65535,arp,in_port="s1-eth2",vlan_tci=0x0000,dl_src=
00:00:00:00:00:02,dl_dst=00:00:00:00:00:01,arp_spa=10.0.0.2,arp_tpa=10.0.0.1,arp_op=2 actions=output:"s1-eth1"
 cookie=0x0, duration=25.850s, table=0, n_packets=5, n_bytes=490, priority=200,ip,nw_src=10.0.0.1,nw_dst=10.0.0.2 actions=drop
```

PROJECT INFO

This project demonstrates how Software Defined Networking (SDN) can be used to control network behavior centrally by simulating packet loss using Mininet and an OpenFlow controller (POX). A custom topology with multiple hosts connected to a switch is created in Mininet, while the controller defines match-action flow rules that selectively drop packets based on source and destination IP addresses. Instead of relying on traditional distributed routing, the controller dynamically installs rules in the switch's flow table, enabling fine-grained control over specific traffic flows (e.g., dropping packets from one host to another while allowing all other communication). The system behavior is verified using tools like ping and Wireshark to observe packet transmission and loss, and further validated through automated regression testing to ensure that drop rules persist and function correctly under different scenarios. Overall, this project highlights SDN's ability to manage network traffic

CODE

drop_controller.py

```
"""

drop_controller.py  (POX version)

================================

POX OpenFlow 1.0 controller for the Packet Drop Simulator.



Place this file inside the POX components directory:

   pox/ext/drop_controller.py



Launch with:

   sudo python3 pox.py log.level --DEBUG drop_controller



Behaviour

---------

* Acts as a learning L2 switch for all traffic by default.
```

```
* Maintains a list of (src_ip, dst_ip) DROP rules.

* On every packet_in, if the IP pair matches a drop rule → silently discard.

* Otherwise → learn MAC, install forwarding flow, forward packet.

* Drop rules are installed as high-priority OpenFlow flow entries

 with NO actions (= drop).


Author : SDN Mininet Project - UE24CS252B

PES2UG24CS577

"""


from pox.core import core

from pox.lib.util import dpid_to_str

import pox.openflow.libopenflow_01 as of

from pox.lib.packet import ethernet, ipv4

from pox.lib.addresses import IPAddr, EthAddr


log = core.getLogger()


# ── Priority levels ──────────────────────────────────────────────────────

PRIORITY_DROP    = 200   # wins over everything

PRIORITY_FORWARD = 100   # learnt forwarding

PRIORITY_FLOOD   =   1   # table-miss fallback




class DropController:

   """

   Per-switch instance that handles OpenFlow events.


   drop_rules : list of (IPAddr, IPAddr) tuples  (src_ip, dst_ip)

   mac_to_port: dict  mac → port

   """


   def __init__(self, connection, drop_rules):

       self.connection  = connection

       self.mac_to_port = {}


       # DROP rule: h2 → h1

       self.drop_rules = drop_rules


       # Listen to OF events on this connection

       connection.addListeners(self)

       log.info("Switch %s connected.", dpid_to_str(connection.dpid))


       # Re-install all current drop rules onto this (possibly reconnected) switch

       for src_ip, dst_ip in self.drop_rules:

           self._install_drop_flow(src_ip, dst_ip)
```

```python
# ── OpenFlow event handlers ──────────────────────────────────────


def _handle_PacketIn(self, event):
    """
    Called for every packet that hits the table-miss rule.
    1. Parse the packet.
    2. Check against drop rules.
    3. Learn MAC → port.
    4. Forward (and optionally install a flow rule).
    """
    packet_data = event.parsed
    if not packet_data.parsed:
        log.warning("Ignoring incomplete packet.")
        return

    in_port = event.port
    dpid    = event.connection.dpid

    eth = packet_data
    src_mac = eth.src
    dst_mac = eth.dst

    # ── DROP check ──────────────────────────────────────────────
    ip_pkt = packet_data.find("ipv4")
    if ip_pkt:
        for (src_ip, dst_ip) in self.drop_rules:
            if ip_pkt.srcip == src_ip and ip_pkt.dstip == dst_ip:
                log.info("[DROP] %s → %s  (packet_in discarded on sw %s)",
                         ip_pkt.srcip, ip_pkt.dstip, dpid_to_str(dpid))
                return   # discard - do not forward

    # ── MAC learning ────────────────────────────────────────────
    self.mac_to_port[src_mac] = in_port

    # ── Forwarding ──────────────────────────────────────────────
    if dst_mac in self.mac_to_port:
        out_port = self.mac_to_port[dst_mac]

        # Install a proactive forwarding flow
        msg             = of.ofp_flow_mod()
        msg.priority    = PRIORITY_FORWARD
        msg.idle_timeout = 30
        msg.hard_timeout = 0
        msg.match        = of.ofp_match.from_packet(packet_data, in_port)
        msg.actions.append(of.ofp_action_output(port=out_port))
        msg.data         = event.ofp          # send buffered packet too
        self.connection.send(msg)
```

```
        else:
            # Destination unknown → flood
            msg = of.ofp_packet_out()
            msg.data = event.ofp
            msg.in_port = in_port
            msg.actions.append(of.ofp_action_output(port=of.OFPP_FLOOD))
            self.connection.send(msg)

    # ── Drop rule management ──────────────────────────────────────────

    def _install_drop_flow(self, src_ip, dst_ip):
        """Push a high-priority DROP flow entry (no actions) to the switch."""
        msg             = of.ofp_flow_mod()
        msg.priority    = PRIORITY_DROP
        msg.hard_timeout = 0
        msg.idle_timeout = 0
        msg.match.dl_type = 0x0800          # IPv4
        msg.match.nw_src  = IPAddr(str(src_ip))
        msg.match.nw_dst  = IPAddr(str(dst_ip))
        # No actions → DROP
        self.connection.send(msg)
        log.info("[RULE INSTALLED] DROP %s → %s on switch %s",
                 src_ip, dst_ip, dpid_to_str(self.connection.dpid))

    def _remove_drop_flow(self, src_ip, dst_ip):
        """Delete a DROP flow entry from the switch."""
        msg             = of.ofp_flow_mod()
        msg.command     = of.OFPFC_DELETE
        msg.priority    = PRIORITY_DROP
        msg.match.dl_type = 0x0800
        msg.match.nw_src  = IPAddr(str(src_ip))
        msg.match.nw_dst  = IPAddr(str(dst_ip))
        msg.out_port      = of.OFPP_NONE
        self.connection.send(msg)
        log.info("[RULE REMOVED] DROP %s → %s on switch %s",
                 src_ip, dst_ip, dpid_to_str(self.connection.dpid))


class DropControllerComponent:
    """
    POX component — manages all switch connections and the shared drop-rule list.

    To add/remove rules from Python (e.g. from the POX interactive shell):
        core.drop_controller.add_drop_rule("10.0.0.1", "10.0.0.2")
        core.drop_controller.remove_drop_rule("10.0.0.1", "10.0.0.2")
    """
```

```python
def __init__(self):
    # Shared drop rules list - (IPAddr, IPAddr) tuples
    self.drop_rules = [
        (IPAddr("10.0.0.2"), IPAddr("10.0.0.1"))
    ]
    # dpid → DropController instance
    self._switches   = {}

    core.openflow.addListeners(self)
    log.info("DropControllerComponent ready.")

# ── POX event: new switch ──────────────────────────────────────────

def _handle_ConnectionUp(self, event):
    sw = DropController(event.connection, self.drop_rules)
    self._switches[event.dpid] = sw

    # FORCE install drop rules after connection
    for src_ip, dst_ip in self.drop_rules:
        sw._install_drop_flow(src_ip, dst_ip)

def _handle_ConnectionDown(self, event):
    self._switches.pop(event.dpid, None)
    log.info("Switch %s disconnected.", dpid_to_str(event.dpid))

# ── Public API ─────────────────────────────────────────────────────

def add_drop_rule(self, src_ip: str, dst_ip: str):
    """
    Add a DROP rule for (src_ip, dst_ip) on ALL connected switches.
    Call from topology.py via:
        from pox.core import core
        core.call_when_ready(lambda: core.drop_controller.add_drop_rule(...),
                             "drop_controller")
    """
    rule = (IPAddr(src_ip), IPAddr(dst_ip))
    if rule not in self.drop_rules:
        self.drop_rules.append(rule)
    for sw in self._switches.values():
        sw._install_drop_flow(*rule)
    log.info("[API] add_drop_rule %s → %s", src_ip, dst_ip)

def remove_drop_rule(self, src_ip: str, dst_ip: str):
    """Remove a DROP rule from the shared list and all connected switches."""
    rule = (IPAddr(src_ip), IPAddr(dst_ip))
    if rule in self.drop_rules:
        self.drop_rules.remove(rule)
```

```
        for sw in self._switches.values():
            sw._remove_drop_flow(*rule)
        log.info("[API] remove_drop_rule %s → %s", src_ip, dst_ip)


    def list_rules(self):
        """Print current drop rules to the log."""
        if not self.drop_rules:
            log.info("[API] No active drop rules.")
        for src, dst in self.drop_rules:
            log.info("[API]  DROP %s → %s", src, dst)




# ── POX entry point ──────────────────────────────────────────────────────


def launch():
    """Called by POX when the component is loaded."""
    component = DropControllerComponent()
    core.register("drop_controller", component)
    log.info("Packet Drop Simulator controller launched.")
```

regression_test.py

```
#!/usr/bin/env python3
"""
regression_test.py  (POX version)
==================================
Standalone regression test suite — does NOT need a running POX controller.
Uses OVS in standalone/learning mode + ovs-ofctl to inject drop rules directly.


Tests
-----
 T1 - Baseline: all 4 hosts can reach each other
 T2 - Drop rule blocks h1 → h2
 T3 - Non-targeted flows are unaffected (h1↔h3, h2↔h4)
 T4 - Removing the rule restores h1 → h2
 T5 - Two simultaneous drop rules both work
 T6 - Drop rule persists after unrelated traffic is generated


Run:
    sudo python3 regression_test.py


Author : SDN Mininet Project - UE24CS252B
"""


import subprocess
import sys
```

```python
import time


from mininet.net import Mininet

from mininet.node import OVSSwitch, Controller

from mininet.topo import Topo

from mininet.log import setLogLevel


# ── Colour helpers ──────────────────────────────────────────────────────

PASS = "\033[92m[PASS]\033[0m"

FAIL = "\033[91m[FAIL]\033[0m"

INFO = "\033[94m[INFO]\033[0m"


results = []   # list of (test_name, passed: bool)




def banner(msg):

   print(f"\n{'─'*60}\n  {msg}\n{'─'*60}")




def record(name, ok, detail=""):

   status = PASS if ok else FAIL

   print(f"  {status}  {name}" + (f"  ({detail})" if detail else ""))

   results.append((name, ok))




# ── Topology ────────────────────────────────────────────────────────────


class FourHostTopo(Topo):

   def build(self):

       s1 = self.addSwitch("s1")

       for i in range(1, 5):

           h = self.addHost(f"h{i}", ip=f"10.0.0.{i}/24")

           self.addLink(h, s1)




def build_net():

   net = Mininet(topo=FourHostTopo(),

                 controller=Controller,   # built-in learning switch

                 switch=OVSSwitch,

                 autoSetMacs=True)

   return net




# ── Rule helpers ────────────────────────────────────────────────────────


def add_drop(sw, src_ip, dst_ip, priority=200):

   subprocess.call(
```

```
        f"ovs-ofctl add-flow {sw} "
        f"\"priority={priority},ip,nw_src={src_ip},nw_dst={dst_ip},actions=drop\"",
        shell=True
    )


def del_drop(sw, src_ip, dst_ip):
    subprocess.call(
        f"ovs-ofctl del-flows {sw} \"ip,nw_src={src_ip},nw_dst={dst_ip}\"",
        shell=True
    )


def dump_flows(sw):
    out = subprocess.check_output(
        f"ovs-ofctl dump-flows {sw}", shell=True, text=True
    )
    print(f"\n{INFO} Flow table ({sw}):\n{out}")


def ping_loss(net, src_name, dst_name, count=4):
    src = net.get(src_name)
    dst = net.get(dst_name)
    out = src.cmd(f"ping -c {count} -W 1 {dst.IP()}")
    for line in out.splitlines():
        if "packet loss" in line:
            for token in line.split():
                if token.endswith("%"):
                    try:
                        return float(token[:-1])
                    except ValueError:
                        pass
    return 100.0


# ── Individual tests ──────────────────────────────────────────────


def t1_baseline(net):
    banner("T1 - Baseline: all hosts reachable")
    l12 = ping_loss(net, "h1", "h2")
    l34 = ping_loss(net, "h3", "h4")
    record("T1a: h1→h2 baseline 0% loss", l12 == 0.0, f"{l12:.0f}%")
    record("T1b: h3→h4 baseline 0% loss", l34 == 0.0, f"{l34:.0f}%")


def t2_drop_blocks(net):
    banner("T2 - Drop rule blocks h1 → h2")
```

```
    add_drop("s1", "10.0.0.1", "10.0.0.2")
    time.sleep(0.3)
    loss = ping_loss(net, "h1", "h2")
    record("T2: h1→h2 blocked (100% loss)", loss == 100.0, f"{loss:.0f}%")


def t3_non_targeted_ok(net):
    banner("T3 - Non-targeted flows unaffected")
    l13 = ping_loss(net, "h1", "h3")
    l24 = ping_loss(net, "h2", "h4")
    record("T3a: h1→h3 still 0% loss", l13 == 0.0, f"{l13:.0f}%")
    record("T3b: h2→h4 still 0% loss", l24 == 0.0, f"{l24:.0f}%")


def t4_remove_restores(net):
    banner("T4 - Removing drop rule restores h1 → h2")
    del_drop("s1", "10.0.0.1", "10.0.0.2")
    time.sleep(1)
    loss = ping_loss(net, "h1", "h2")
    record("T4: h1→h2 restored (0% loss)", loss == 0.0, f"{loss:.0f}%")


def t5_multiple_rules(net):
    banner("T5 - Multiple simultaneous drop rules")
    add_drop("s1", "10.0.0.1", "10.0.0.2")
    add_drop("s1", "10.0.0.3", "10.0.0.4")
    time.sleep(0.3)

    l12 = ping_loss(net, "h1", "h2")
    l34 = ping_loss(net, "h3", "h4")
    l13 = ping_loss(net, "h1", "h3")   # should be unaffected

    record("T5a: h1→h2 dropped",       l12 == 100.0, f"{l12:.0f}%")
    record("T5b: h3→h4 dropped",       l34 == 100.0, f"{l34:.0f}%")
    record("T5c: h1→h3 unaffected",    l13 == 0.0,   f"{l13:.0f}%")

    del_drop("s1", "10.0.0.1", "10.0.0.2")
    del_drop("s1", "10.0.0.3", "10.0.0.4")


def t6_persistence(net):
    banner("T6 - Drop rule persists after unrelated traffic")
    add_drop("s1", "10.0.0.2", "10.0.0.3")
    time.sleep(0.3)

    # Generate unrelated traffic
    net.get("h1").cmd("ping -c 4 10.0.0.4 &")
```

```
    net.get("h4").cmd("ping -c 4 10.0.0.1 &")
    time.sleep(2)

    loss = ping_loss(net, "h2", "h3")
    record("T6: h2→h3 rule persists after other traffic",
           loss == 100.0, f"{loss:.0f}%")
    del_drop("s1", "10.0.0.2", "10.0.0.3")


# ── Entry point ──────────────────────────────────────────────────────

def main():
    setLogLevel("warning")
    print("\n" + "=" * 60)
    print("  Packet Drop Simulator - Regression Test Suite (POX)")
    print("=" * 60)

    net = build_net()
    net.start()
    time.sleep(2)

    try:
        t1_baseline(net)
        t2_drop_blocks(net)
        t3_non_targeted_ok(net)
        t4_remove_restores(net)
        t5_multiple_rules(net)
        t6_persistence(net)
        dump_flows("s1")
    finally:
        net.stop()

    banner("TEST SUMMARY")
    passed = sum(1 for _, ok in results if ok)
    total  = len(results)
    for name, ok in results:
        print(f"  {'✓' if ok else '✗'}  {name}")
    print(f"\n  Result: {passed}/{total} tests passed")
    print("=" * 60 + "\n")
    sys.exit(0 if passed == total else 1)


if __name__ == "__main__":
    main()
```

topology.py

```
#!/usr/bin/env python3

"""

topology.py  (POX version)

==========================

Mininet topology for the Packet Drop Simulator.


Network layout

--------------

   h1 (10.0.0.1) ──┐

   h2 (10.0.0.2) ──┤

                   s1 ── RemoteController (POX, port 6633)

   h3 (10.0.0.3) ──┤

   h4 (10.0.0.4) ──┘


Usage

-----

   # Just open CLI (manual testing)

   sudo python3 topology.py


   # Run all automated test scenarios

   sudo python3 topology.py --run-tests


   # Install one drop rule then open CLI

   sudo python3 topology.py --drop-src 10.0.0.1 --drop-dst 10.0.0.2


NOTE: Start POX controller FIRST in another terminal:

   cd ~/pox

   sudo python3 pox.py log.level --DEBUG openflow.of_01 drop_controller


Author : SDN Mininet Project - UE24CS252B

"""


import argparse

import subprocess

import time

import sys


from mininet.net import Mininet

from mininet.node import RemoteController, OVSSwitch

from mininet.topo import Topo

from mininet.log import setLogLevel, info

from mininet.cli import CLI

from mininet.link import TCLink
```

```python
# ── Topology ──────────────────────────────────────────────────────────


class DropSimTopo(Topo):
    """Single OVS switch with 4 hosts and bandwidth-limited links."""

    def build(self):
        s1 = self.addSwitch("s1")
        for i in range(1, 5):
            h = self.addHost(f"h{i}", ip=f"10.0.0.{i}/24",
                             mac=f"00:00:00:00:00:0{i}")
            # 10 Mbps, 5 ms delay - gives realistic ping RTTs for measurement
            self.addLink(h, s1, bw=10, delay="5ms")


# ── Utility helpers ───────────────────────────────────────────────────


def banner(text):
    print("\n" + "=" * 60)
    print(f"  {text}")
    print("=" * 60)


def ovs_add_drop(sw_name, src_ip, dst_ip, priority=200):
    """Install DROP rule using ovs-ofctl (works independently of controller)."""
    cmd = (
        f"ovs-ofctl add-flow {sw_name} "
        f"\"priority={priority},ip,nw_src={src_ip},nw_dst={dst_ip},actions=drop\""
    )
    subprocess.call(cmd, shell=True)
    info(f"  [RULE] DROP installed: {src_ip} → {dst_ip}\n")


def ovs_del_drop(sw_name, src_ip, dst_ip):
    """Remove a DROP rule using ovs-ofctl."""
    cmd = (
        f"ovs-ofctl del-flows {sw_name} "
        f"\"ip,nw_src={src_ip},nw_dst={dst_ip}\""
    )
    subprocess.call(cmd, shell=True)
    info(f"  [RULE] DROP removed: {src_ip} → {dst_ip}\n")


def dump_flows(sw_name):
    """Print the current flow table."""
    out = subprocess.check_output(
```

```
        f"ovs-ofctl dump-flows {sw_name}", shell=True, text=True
    )
    info(f"\n--- Flow table ({sw_name}) ---\n{out}\n")


def ping_loss(net, src_name, dst_name, count=5):
    """Ping from src to dst; return packet-loss percentage."""
    src = net.get(src_name)
    dst = net.get(dst_name)
    result = src.cmd(f"ping -c {count} -W 1 {dst.IP()}")
    loss = 100.0
    for line in result.splitlines():
        if "packet loss" in line:
            for token in line.split():
                if token.endswith("%"):
                    try:
                        loss = float(token[:-1])
                    except ValueError:
                        pass
    info(f"  [{src_name} → {dst_name}] Packet loss: {loss:.0f}%\n")
    return loss


def run_iperf(net, src_name, dst_name, duration=5):
    """Measure TCP throughput with iperf."""
    src = net.get(src_name)
    dst = net.get(dst_name)
    info(f"  iperf [{src_name} → {dst_name}] {duration}s …\n")
    dst.cmd("iperf -s -D")
    time.sleep(0.5)
    out = src.cmd(f"iperf -c {dst.IP()} -t {duration}")
    dst.cmd("kill %iperf")
    for line in out.splitlines():
        if "bits/sec" in line:
            info(f"    {line.strip()}\n")


# ── Test scenarios ──────────────────────────────────────────────


def scenario_1_baseline(net):
    banner("SCENARIO 1 - Baseline: Normal Forwarding (no drop rules)")
    net.pingAll()
    info("\n[Expected] 0% packet loss for all pairs.\n")


def scenario_2_drop(net):
    banner("SCENARIO 2 - Drop Rule: h1 (10.0.0.1) → h2 (10.0.0.2)")
```

```
    ovs_add_drop("s1", "10.0.0.1", "10.0.0.2")
    dump_flows("s1")

    info("\n[Blocked flow h1 → h2 - expect 100% loss]:\n")
    loss_blocked = ping_loss(net, "h1", "h2")

    info("\n[Allowed flow h1 → h3 - expect 0% loss]:\n")
    loss_allowed = ping_loss(net, "h1", "h3")

    assert loss_blocked == 100.0, \
        f"FAIL: expected 100% loss h1→h2, got {loss_blocked}%"
    assert loss_allowed == 0.0, \
        f"FAIL: expected 0% loss h1→h3, got {loss_allowed}%"
    info("\n[PASS] Scenario 2 assertions passed.\n")
    return loss_blocked, loss_allowed


def scenario_3_regression(net):
    banner("SCENARIO 3 - Regression: Rule Persistence")

    # 3a - rule from scenario 2 should still be active
    info("  [3a] h1→h2 drop rule still active?\n")
    loss = ping_loss(net, "h1", "h2")
    assert loss == 100.0, f"FAIL: rule should persist, got {loss}%"
    info("  [PASS] Rule persisted.\n")

    # 3b - add second drop rule
    info("  [3b] Adding second drop rule: h3 → h4\n")
    ovs_add_drop("s1", "10.0.0.3", "10.0.0.4")
    loss2 = ping_loss(net, "h3", "h4")
    assert loss2 == 100.0, f"FAIL: h3→h4 should be dropped, got {loss2}%"
    info("  [PASS] Second drop rule works.\n")

    # 3c - remove first rule, h1→h2 should recover
    info("  [3c] Removing h1→h2 rule - traffic should restore\n")
    ovs_del_drop("s1", "10.0.0.1", "10.0.0.2")
    time.sleep(1)
    loss3 = ping_loss(net, "h1", "h2")
    assert loss3 == 0.0, f"FAIL: h1→h2 should recover, got {loss3}%"
    info("  [PASS] Traffic restored after rule removal.\n")

    dump_flows("s1")
    info("\n[PASS] All regression checks passed.\n")


def scenario_4_iperf(net):
    banner("SCENARIO 4 - Throughput Measurement with iperf")
```

```
    info("  [Allowed flow h1 → h3]:\n")
    run_iperf(net, "h1", "h3", duration=5)

    info("  [Dropped flow h3 → h4 - expect 0 throughput / connection fail]:\n")
    h3 = net.get("h3")
    h4 = net.get("h4")
    h4.cmd("iperf -s -D")
    time.sleep(0.5)
    out = h3.cmd(f"iperf -c {h4.IP()} -t 5 2>&1")
    h4.cmd("kill %iperf")
    # Print first 300 chars to avoid noise
    info(f"    {out[:300]}\n")


# ── Main ──────────────────────────────────────────────────────────────

def main():
    parser = argparse.ArgumentParser(
        description="Packet Drop Simulator - Mininet (POX)"
    )
    parser.add_argument("--controller-ip",   default="127.0.0.1")
    parser.add_argument("--controller-port", type=int, default=6633)
    parser.add_argument("--run-tests",  action="store_true",
                        help="Run all automated scenarios")
    parser.add_argument("--drop-src",   default=None,
                        help="Source IP for a manual drop rule")
    parser.add_argument("--drop-dst",   default=None,
                        help="Destination IP for a manual drop rule")
    args = parser.parse_args()

    setLogLevel("info")

    topo       = DropSimTopo()
    controller = RemoteController("c0",
                                  ip=args.controller_ip,
                                  port=args.controller_port)
    net = Mininet(topo=topo,
                  controller=controller,
                  switch=OVSSwitch,
                  link=TCLink,
                  autoSetMacs=False)

    info("\n*** Starting Packet Drop Simulator (POX) ***\n")
    net.start()

    info("Waiting for controller handshake …\n")
    time.sleep(3)
```

```
    try:
        if args.run_tests:
            scenario_1_baseline(net)
            scenario_2_drop(net)
            scenario_3_regression(net)
            scenario_4_iperf(net)
            banner("ALL SCENARIOS COMPLETE")
        elif args.drop_src and args.drop_dst:
            ovs_add_drop("s1", args.drop_src, args.drop_dst)
            dump_flows("s1")

        CLI(net)
    finally:
        net.stop()


if __name__ == "__main__":
    main()
```